NORAIZ RANA

Task #25

Topic

Introduction to React Query for Data Fetching

Date:

18 August 2025

SUBMITTED TO:

APPVERSE TECHNOLOGIES

📄 Learning Report: Introduction to React Query for Data Fetching

1. Introduction

React Query is a **data-fetching and state management library** for React applications. Instead of manually handling `useEffect`, `useState`, loading states, and error management, React Query simplifies these tasks. It helps developers **fetch, cache, update, and synchronize data** in a very efficient way.

Traditional API handling with `fetch` or `axios` requires writing separate logic for **loading, error, and data states**. React Query automates most of this, making the code cleaner and more maintainable.

---

2. Core Features of React Query

- **Declarative Data Fetching** – Simply define how data should be fetched, and React Query handles when and how often.
- **Built-in Loading & Error States** – No need to write extra `useState` for these.
- **Caching** – Data is cached, reducing unnecessary API calls.
- **Refetching** – Data can automatically refresh when the app regains focus or reconnects to the internet.
- **Background Updates** – Users see old data until new data is fetched.
- **Pagination & Infinite Queries** – Makes handling lists or scrollable content easier.

---

3. Basic Usage Example

```
import { useQuery } from '@tanstack/react-
query';

function Users() {
  const { data, isLoading, error } = useQuery({
    queryKey: ['users'],
    queryFn: () =>
fetch('https://api.example.com/users').then(res
=> res.json())
  });

  if (isLoading) return <p>Loading...</p>;
  if (error) return <p>Error:
{error.message}</p>;

  return (
    <ul>
      {data.map(user => <li
key={user.id}>{user.name}</li>)}
    </ul>
  );
}
```

Here:

- `isLoading` replaces our manual loading state.
- `error` gives us error details automatically.
- `data` contains API response.

---

4. Benefits Over Traditional Fetching

- **Less Boilerplate:** No need to manually manage multiple states.
- **Performance Optimized:** Uses caching and stale-while-revalidate strategy.

- **User Experience:** Old data remains visible while new data loads.
- **Auto Refetching:** Keeps data fresh without extra code.

React Query allows developers to focus on **application logic** instead of managing repetitive state transitions, making applications more **scalable, performant, and user-friendly**.